

Visualizing Data with Pandas

From DataFrames to Graphs

The Pandas-Matplotlib Connection

Pandas plotting is built on top of matplotlib

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('data/tuneindex.csv')
```

- Pandas provides the **convenience** (simple syntax)
- Matplotlib provides the **power** (full customization)
- They work together seamlessly

"Pandas gets you 80% of the way, matplotlib takes you the rest"

Your First Plot

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('data/tuneindex.csv')

# Count tune types and plot
tune_counts = df['tune_type'].value_counts()
tune_counts.plot(kind='bar')
plt.show()
```

That's it! Three lines and you have a visualization.

The .plot() Method

Every pandas DataFrame and Series has a `.plot()` method

```
# These all work:  
df.plot()                # Default line plot  
df['column'].plot()      # Plot one column  
df.plot(x='col1', y='col2') # Specify x and y
```

Plot types:

- `plot.bar()` - Bar chart
- `plot.line()` - Line chart
- `plot.scatter()` - Scatter plot
- `plot.hist()` - Histogram
- `plot.box()` - Box plot

Example: Tune Type Distribution

Question: Which tune types are most common?

```
# Count and visualize
type_counts = df['tune_type'].value_counts()

type_counts.plot.bar(figsize=(10, 6))
plt.title('Distribution of Tune Types')
plt.xlabel('Tune Type')
plt.ylabel('Number of Tunes')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Notice: pandas creates the plot, matplotlib customizes it

Adding Matplotlib Customization

```
# Create the plot (pandas)
ax = df['tune_type'].value_counts().plot.bar(figsize=(10, 6))

# Customize it (matplotlib)
ax.set_title('Tune Types in Irish Traditional Music', fontsize=16)
ax.set_xlabel('Tune Type', fontsize=12)
ax.set_ylabel('Count', fontsize=12)
ax.grid(axis='y', alpha=0.3)

plt.show()
```

The `.plot()` method returns a matplotlib **Axes object** that you can customize

Key Signature Popularity

```
# Top 10 most common keys
top_keys = df['key_sig'].value_counts().head(10)

ax = top_keys.plot.barh(figsize=(8, 6), color='steelblue')
ax.set_title('Top 10 Most Common Keys')
ax.set_xlabel('Number of Tunes')
ax.invert_yaxis() # Most popular at top
plt.tight_layout()
plt.show()
```

Note: `.barh()` creates horizontal bars - great for long labels!

Histogram: Download Distribution

Question: How are downloads distributed?

```
# Look at download patterns
df['downloaded'].plot.hist(bins=50, figsize=(10, 6))
plt.title('Distribution of Tune Popularity')
plt.xlabel('Number of Downloads')
plt.ylabel('Frequency')
plt.axvline(df['downloaded'].median(),
            color='red', linestyle='--',
            label=f'Median: {df["downloaded"].median():.0f}')
plt.legend()
plt.show()
```


Comparing Groups

Question: Do reels get more downloads than jigs?

```
# Get data for each type
reels = df[df['tune_type'] == 'reel']['downloaded']
jigs = df[df['tune_type'] == 'jig']['downloaded']

# Create comparison
comparison = pd.DataFrame({
    'Reels': reels.values,
    'Jigs': jigs.values[:len(reels)] # Match lengths
})

comparison.plot.hist(bins=30, alpha=0.7, figsize=(10, 6))
plt.title('Download Distribution: Reels vs Jigs')
plt.xlabel('Downloads')
plt.legend()
plt.show()
```

Box Plot Comparison

Better way to compare distributions:

```
# Compare all tune types at once
df.boxplot(column='downloaded', by='tune_type', figsize=(12, 6))
plt.suptitle('') # Remove default title
plt.title('Download Distribution by Tune Type')
plt.xlabel('Tune Type')
plt.ylabel('Downloads')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Box plots show median, quartiles, and outliers!

Grouped Bar Charts

Question: Which keys are most popular for each tune type?

```
# Group by tune type and key, count them
grouped = df.groupby(['tune_type', 'key_sig']).size().unstack(fill_value=0)

# Plot top keys for each type
grouped.head(5).plot.bar(figsize=(12, 6))
plt.title('Top Keys by Tune Type')
plt.xlabel('Tune Type')
plt.ylabel('Number of Tunes')
plt.legend(title='Key Signature', bbox_to_anchor=(1.05, 1))
plt.tight_layout()
plt.show()
```

Average Popularity by Key

```
# Calculate average downloads for each key
avg_by_key = df.groupby('key_sig')['downloaded'].mean().sort_values(ascending=False).head(10)

ax = avg_by_key.plot.bar(figsize=(10, 6), color='darkgreen')
ax.set_title('Average Downloads by Key Signature')
ax.set_ylabel('Average Downloads')
ax.set_xlabel('Key Signature')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Scatter Plot: Exploring Relationships

Question: Does anything predict popularity?

```
# If we had settings count data
ax = df.plot.scatter(x='id', y='downloaded',
                    alpha=0.5, figsize=(10, 6))
ax.set_title('Tune ID vs Popularity')
ax.set_xlabel('Tune ID (chronological)')
ax.set_ylabel('Downloads')
plt.show()
```

Lower alpha (transparency) helps with overlapping points

The Complete Workflow

```
import pandas as pd
import matplotlib.pyplot as plt

# 1. Load data
df = pd.read_csv('data/tuneindex.csv')

# 2. Analyze with pandas
popular_types = df.groupby('tune_type')['downloaded'].mean().sort_values(ascending=False)

# 3. Create plot with pandas
ax = popular_types.plot.bar(figsize=(10, 6), color='steelblue')

# 4. Customize with matplotlib
ax.set_title('Average Popularity by Tune Type', fontsize=14, pad=20)
ax.set_ylabel('Average Downloads', fontsize=12)
ax.grid(axis='y', alpha=0.3)

# 5. Show it
plt.tight_layout()
plt.show()
```

Common Plot Patterns

Categorical Data → Bar Chart

```
df['category'].value_counts().plot.bar()
```

Numerical Distribution → Histogram

```
df['numbers'].plot.hist(bins=30)
```

Comparing Groups → Box Plot

```
df.boxplot(column='value', by='group')
```

Two Variables → Scatter Plot

```
df.plot.scatter(x='var1', y='var2')
```

Saving Your Plots

Don't just show - save them!

```
# Create your plot
df['tune_type'].value_counts().plot.bar()
plt.title('Tune Types Distribution')

# Save before showing
plt.savefig('tune_types.png', dpi=300, bbox_inches='tight')
plt.show()
```

Formats: `.png`, `.jpg`, `.pdf`, `.svg`

Subplots: Multiple Plots

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Four different views of the data
df['tune_type'].value_counts().plot.bar(ax=axes[0, 0])
axes[0, 0].set_title('Tune Types')

df['key_sig'].value_counts().head(5).plot.bar(ax=axes[0, 1])
axes[0, 1].set_title('Top Keys')

df['downloaded'].plot.hist(bins=30, ax=axes[1, 0])
axes[1, 0].set_title('Download Distribution')

df.boxplot(column='downloaded', by='tune_type', ax=axes[1, 1])
axes[1, 1].set_title('Downloads by Type')

plt.tight_layout()
plt.show()
```

Styling Your Plots

Pandas respects matplotlib style sheets:

```
# Use a different style
plt.style.use('seaborn-v0_8-darkgrid') # or 'ggplot', 'fivethirtyeight'

df['tune_type'].value_counts().plot.bar(figsize=(10, 6))
plt.title('Styled Plot')
plt.show()
```

Try different styles to find what looks best!

Common Pitfalls

Forgetting plt.show()

```
df.plot.bar()    # Nothing appears!  
plt.show()       # Now it shows
```

Not sizing appropriately

```
# Too small - use figsize  
df.plot.bar(figsize=(10, 6)) # Much better
```

Overlapping labels

```
plt.xticks(rotation=45) # Angle them  
plt.tight_layout()      # Fit everything
```

Quick Reference

```
# Basic pattern
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('data.csv')
df['column'].plot.TYPE(figsize=(10, 6))
plt.title('My Title')
plt.xlabel('X Label')
plt.ylabel('Y Label')
plt.show()
```

Replace TYPE with: bar, line, hist, box, scatter, barh

Practice Exercise 1

Task: Create a bar chart showing the top 5 most downloaded tunes

```
# Your code here:  
top5 = df.nlargest(5, 'downloaded')  
top5.plot.bar(x='title', y='downloaded', figsize=(10, 6))  
plt.title('Top 5 Most Downloaded Tunes')  
plt.ylabel('Downloads')  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```

Practice Exercise 2

Task: Compare download distributions for reels and jigs using box plots

```
# Your code here:  
reel_jig = df[df['tune_type'].isin(['reel', 'jig'])]  
reel_jig.boxplot(column='downloaded', by='tune_type', figsize=(8, 6))  
plt.suptitle('')  
plt.title('Downloads: Reels vs Jigs')  
plt.show()
```

Key Takeaways

1. **Pandas does the data work** - filtering, grouping, counting
2. **Pandas creates quick plots** - `.plot()` method
3. **Matplotlib adds polish** - titles, labels, styling
4. **They work together** - pandas returns matplotlib objects
5. **Start simple, add complexity** - basic plot first, then customize

Next Steps

Now you can:

-  Create basic visualizations from DataFrames
-  Customize plots with matplotlib
-  Choose the right plot type for your data
-  Compare different groups visually
-  Save and share your visualizations

Practice with your tune data!

Resources

Documentation:

- Pandas plotting: pandas.pydata.org/docs/user_guide/visualization.html
- Matplotlib gallery: matplotlib.org/stable/gallery/index.html

In your code:

```
help(df.plot) # See all plotting options
```

Try different things - visualization is learned by doing!

Questions to Explore

Using your new plotting skills:

- Which key signatures are overrepresented in popular tunes?
- Is there a "golden age" in the tune ID range?
- Do hornpipes have different download patterns than reels?
- What's the relationship between tune type and key choice?

Your turn to investigate! 🎵📊